

Reference Manual — Trusted Agentic Commerce for Magento 2

Version 1.1.1 · Technical reference for developers and system integrators

Table of contents

1. [Module architecture](#)
 2. [Configuration reference](#)
 3. [Database schema](#)
 4. [Events and observers](#)
 5. [Cron jobs](#)
 6. [ACL resources](#)
 7. [Admin routes](#)
 8. [Frontend routes](#)
 9. [Services](#)
 10. [Enforcement engine](#)
 11. [Webhook outbox](#)
 12. [Agent token verification](#)
 13. [MCP manifest](#)
 14. [Extension attributes](#)
 15. [API endpoints called](#)
 16. [Security model](#)
 17. [Console commands](#)
 18. [Data patches](#)
 19. [Di.xml wiring](#)
 20. [Logging](#)
-

1. Module architecture

```
Trusted_AgenticCommerce
├─ Block/Adminhtml/      SPA placeholder blocks for admin pages
├─ Console/Command/      CLI commands (check-webserver, webhook:status)
├─ Controller/
│   ├─ Adminhtml/        Admin controllers (Dashboard, Setup, Health, etc.)
│   └─ Wellknown/         Frontend /.well-known/mcp.json endpoint
├─ Cron/                 Webhook drainer + lag heartbeat
├─ Enforcement/          R043 HITL gate logic
├─ Model/
│   ├─ Config/            ScopeConfig readers + SSRF validator
│   ├─ Manifest/          MCP manifest builder
│   ├─ Security/          HMAC signer + API URL validator
│   ├─ Setup/             Setup wizard data provider
│   ├─ Storefront/        Hyvä/PWA theme detector
│   └─ Webhook/           Outbox repository + signature publisher
├─ Observer/             Magento event hooks (order, shipment, payment)
├─ Plugin/               Order extension attribute plugin
├─ Router/               /.well-known URL router
├─ Service/              EnforcementClient, AgentTokenVerifier, CartSignals
├─ Setup/Patch/Data/     Data patches (EAV attribute, Hyvä bridge, install event)
├─ Test/
│   ├─ Integration/       Integration test suite
│   ├─ Static/            Route consistency checker
│   └─ Unit/              Unit test suite
├─ etc/                  Module XML, DI, ACL, events, cron, system config
├─ i18n/                 Translation files
└─ view/                 Admin layout + templates
```

Module name: `Trusted_AgenticCommerce` **Composer package:** `trusteded/agentic-commerce-magento` **PHP namespace:** `Trusteded\AgenticCommerce` **Setup version:** none (uses Data Patches exclusively)

2. Configuration reference

All paths are under `Stores → Configuration → Trusteded → Agentic Commerce` (`trusteded_general` section in `system.xml`).

Group: API Connection (`trusted_general/general`)

Field	Config Path	Type	Scope	Description
API Base URL	<code>trusted_general/general/api_base_url</code>	text	Global	Trusted API root (must be HTTPS). Default: <code>https://api.trusted.xyz</code>
Merchant ID	<code>trusted_general/general/merchant_id</code>	text	Global	Assigned by Trusted at account creation
Integration Token	<code>trusted_general/general/integration_token</code>	obscure	Global	Bearer token for outbound API calls. Stored encrypted via <code>Magento\Config\Model\Config\Backend\Encrypted</code>
Webhook Secret	<code>trusted_general/general/webhook_secret</code>	obscure	Global	HMAC-SHA256 secret for signing webhook deliveries. Stored encrypted
Internal HMAC Secret	<code>trusted_general/general/internal_hmac_secret</code>	obscure	Global	Signs internal heartbeat calls with the <code>X-Trusted-Connection-Id</code> , <code>X-Trusted-Timestamp</code> and <code>X-Trusted-Signature</code> headers. The older <code>X-Internal-Auth</code> scheme is rejected by the Trusted API
Webhook Secret Version	<code>trusted_general/general/webhook_secret_version</code>	text	Global	Increment when rotating the webhook secret. Used in the <code>X-Trusted-Webhook-Secret-Version</code> header
Connection ID	<code>trusted_general/general/connection_id</code>	text	Website	Assigned by Trusted after connecting. Identifies this store in webhook delivery

Group: Features (`trusted_general/features`)

Field	Config Path	Type	Scope	Description
Enable WebMCP Bridge	<code>trusted_general/features/webmcp_enabled</code>	select	Website	Injects storefront JS bridge. Auto-disabled on Hyvä/PWA Studio via <code>DisableBridgeOnHyva</code> patch
Enable Phase B	<code>trusted_general/features/phase_b_enabled</code>	select	Global	Reserved for future embedded SPA. Do not enable

Enforcement paths

Config Path	Description
<code>trusted/enforcement/failure_mode</code>	<code>observe</code> or <code>enforce</code> . Default: <code>enforce</code> . You can change it in Trusted → Adjustes
<code>trusted/enforcement/installation_id</code>	Enforcement Installation ID that Trusted provides. You enter it in the Setup Wizard
<code>trusted/enforcement/hmac_secret</code>	Enforcement HMAC Secret that Trusted provides, used to sign <code>POST /v1/rules/evaluate</code> . You enter it in the Setup Wizard

3. Database schema

Table: `trusted_webhook_outbox`

Reliable delivery outbox for order events. Entries are created by observers and drained every minute by the `trusted_webhook_drain` cron job.

Column	Type	Nullable	Description
<code>id</code>	int unsigned AUTO_INCREMENT	No	Primary key
<code>event_id</code>	varchar(36) UNIQUE	No	UUID v4 event identifier (idempotency key)
<code>event_type</code>	varchar(32)	No	<code>order_created</code> , <code>order_completed</code> , <code>order_refunded</code> , <code>order_cancelled</code> , <code>order_fulfilled</code>
<code>entity_id</code>	int unsigned	No	Magento order <code>entity_id</code>
<code>increment_id</code>	varchar(32)	No	Magento order increment ID (e.g. <code>000000001</code>)
<code>composite_id</code>	varchar(64)	No	Composite ID in format <code>MAG:<entity_id></code>
<code>store_view_code</code>	varchar(32)	No	Store view code at time of event
<code>payload</code>	text	No	Serialized order payload (JSON)
<code>status</code>	varchar(16)	No	<code>pending</code> , <code>delivered</code> , <code>dead</code>
<code>retry_count</code>	int unsigned	No	Delivery attempt count. Default: 0
<code>secret_version</code>	int unsigned	No	Webhook secret version at time of enqueue
<code>next_attempt_at</code>	timestamp	Yes	Earliest eligible delivery time (exponential backoff). NULL = eligible immediately
<code>locked_until</code>	timestamp	Yes	Lease lock expiry (worker lease mechanism)
<code>locked_by</code>	varchar(64)	Yes	Process identifier holding the lease
<code>created_at</code>	timestamp	No	Entry creation time
<code>updated_at</code>	timestamp	No	Last modification time (auto- updated)

Indexes: - `TRUSTEED_WEBHOOK_OUTBOX_EVENT_ID` (UNIQUE) on `event_id` -
`TRUSTEED_WEBHOOK_OUTBOX_STATUS_CREATED_AT` on `(status, created_at)` -
`TRUSTEED_WEBHOOK_OUTBOX_LOCKED_UNTIL` on `locked_until` -
`TRUSTEED_WEBHOOK_OUTBOX_STATUS_NEXT_ATTEMPT` on `(status, next_attempt_at, created_at)`

Columns added to `sales_order`

Column	Type	Description
<code>trusted_receipt_uri</code>	<code>varchar(1024)</code> nullable	TrustReceipt URI. Immutable once set
<code>trusted_receipt_status</code>	<code>varchar(16)</code> nullable	<code>signed</code> . Set when the receipt URI is first stored

4. Events and observers

Magento Event	Observer Class	Purpose
<code>sales_order_save_after</code>	<code>SalesOrderSaveAfter</code>	Enqueues <code>order_created</code> (new, processing and on-hold orders), <code>order_completed</code> , <code>order_refunded</code> (closed orders) and <code>order_cancelled</code> events to the outbox based on order state
<code>sales_order_creditmemo_save_after</code>	<code>SalesCreditmemoSaveAfter</code>	Enqueues <code>order_refunded</code> for partial refunds (full refunds handled via <code>STATE_CLOSED</code> in <code>sales_order_save_after</code>)
<code>sales_model_service_quote_submit_before</code>	<code>CheckoutSubmitBefore</code>	Pre-order enforcement: verifies agent token, calls <code>/v1/rules/evaluate</code> , applies HITL freeze (R043) or throws <code>LocalizedException</code> on BLOCK
<code>sales_order_payment_failed</code>	<code>SalesOrderPaymentFailedObserver</code>	Sends an HMAC-signed checkout-failure signal to <code>/api/v1/checkout-failures</code> for R011 tracking. It does not use the outbox
<code>sales_order_shipment_save_after</code>	<code>ShipmentSaveAfter</code>	Enqueues <code>order_fulfilled</code> when a shipment is created

Plugin

Plugin Class	Target	Method	Type	Purpose
<code>OrderRepositoryPlugin</code>	<code>Magento\Sales\Api\OrderRepositoryInterface</code>	<code>get</code>	after	Loads <code>trusted_r</code> and <code>trusted_r</code> extension attributes on order load
<code>OrderExtensionAttribute</code>	<code>Magento\Sales\Api\OrderRepositoryInterface</code>	<code>save</code>	around	Persists extension attributes on

5. Cron jobs

Both jobs are in the `default` cron group and run every minute.

`trusted_webhook_drain`

Class: `Trusteed\AgenticCommerce\Cron\DrainOutbox`

Processes the `trusted_webhook_outbox` table:

1. Selects entries where `status = 'pending'` AND (`next_attempt_at IS NULL` OR `next_attempt_at ≤ NOW()`) AND (`locked_until IS NULL` OR `locked_until < NOW()`)
2. Acquires a lease (`locked_by = <worker-id>` , `locked_until = NOW() + 60s`)
3. POSTs the payload to `POST /api/v1/webhook/magento/<connection_id>` with the signature headers described in [Webhook outbox](#)
4. On success (2xx): sets `status = 'delivered'`
5. On failure: increments `retry_count` , sets an exponential `next_attempt_at` ($2 \times \text{retry_count}$, capped at 1 hour, with $\pm 20\%$ jitter)
6. After 8 retries: sets `status = 'dead'`

`trusted_lag_heartbeat`

Class: `Trusteed\AgenticCommerce\Cron\EmitLagHeartbeat`

Every minute, calculates the age of the oldest `pending` outbox entry and reports it to `POST /api/v1/internal/magento/lag-heartbeat` so the Trusteed dashboard can alert on delivery lag.

6. ACL resources

Resource ID	Title	Notes
<code>Trusted_AgenticCommerce::config</code>	Trusted Agentic Commerce	Grants access to all Trusted admin pages
<code>Trusted_AgenticCommerce::token</code>	Trusted Embed Token Relay	Grants access to the token issue controller used by the Setup Wizard

To grant a custom role access to Trusted pages, add `Trusted_AgenticCommerce::config` in **System → Permissions → User Roles → [Role] → Role Resources**.

7. Admin routes

Front name: `trusted` (defined in `etc/adminhtml/routes.xml`)

URL Pattern	Controller	Description
<code>/trusteed/dashboard/index</code>	<code>Controller/Adminhtml/</code> <code>Dashboard/Index</code>	Dashboard SPA host
<code>/trusteed/health/index</code>	<code>Controller/Adminhtml/</code> <code>Health/Index</code>	Store health SPA
<code>/trusteed/ventas/index</code>	<code>Controller/Adminhtml/</code> <code>Ventas/Index</code>	Agent sales SPA
<code>/trusteed/agentes/index</code>	<code>Controller/Adminhtml/</code> <code>Agentes/Index</code>	Agent directory SPA
<code>/trusteed/reglas/index</code>	<code>Controller/Adminhtml/</code> <code>Reglas/Index</code>	Rules SPA
<code>/trusteed/pagos/index</code>	<code>Controller/Adminhtml/</code> <code>Pagos/Index</code>	Payment methods SPA
<code>/trusteed/seguridad/index</code>	<code>Controller/Adminhtml/</code> <code>Seguridad/Index</code>	Security SPA
<code>/trusteed/ajustes/index</code>	<code>Controller/Adminhtml/</code> <code>Ajustes/Index</code>	Settings SPA
<code>/trusteed/setup/wizard</code>	<code>Controller/Adminhtml/</code> <code>Setup/Wizard</code>	Setup Wizard
<code>/trusteed/setup/save</code>	<code>Controller/Adminhtml/</code> <code>Setup/Save</code>	Setup Wizard save action
<code>/trusteed/setup/introspecttoken</code>	<code>Controller/Adminhtml/</code> <code>Setup/IntrospectToken</code>	Token introspection AJAX
<code>/trusteed/token/issue</code>	<code>Controller/Adminhtml/</code> <code>Token/Issue</code>	Issues embed token (POST)
<code>/trusteed/support/submit</code>	<code>Controller/Adminhtml/</code> <code>Support/Submit</code>	Support form submit

8. Frontend routes

Front name: `nlweb` (defined in `etc/frontend/routes.xml`)

URL Pattern	Controller	Description
<code>/nlweb/products/index</code>	<code>Controller/Products/Index</code>	NLWeb product search endpoint (proxied)
<code>/nlweb/wellknown/mcpmanifest</code>	<code>Controller/Wellknown/McpManifest</code>	Returns the MCP manifest JSON

The `/.well-known/mcp.json` canonical URL is handled by `Router/WellKnownRouter.php`, which maps `/.well-known/mcp.json` to `nlweb/wellknown/mcpmanifest`.

9. Services

EnforcementClient

Class: `Trusted\AgenticCommerce\Service\EnforcementClient`

HTTP client for the Trusted rule evaluation API.

Method	Returns	Description
<code>evaluate(array \$payload): string</code>	<code>ALLOW</code> <code>BLOCK</code> <code>ESCALATE</code>	Calls <code>POST /v1/rules/evaluate</code> . On transport error, tries the offline safety valve first, then returns <code>BLOCK</code> (enforce mode) or <code>ALLOW</code> (observe mode)
<code>getDidResolver(string \$merchantId): array</code>	<code>array<{did, publicKeyJwk}></code>	Fetches agent DID → public key map from the snapshot. Empty array on failure
<code>getRules(string \$merchantId): array</code>	<code>array<{ruleCode, params, mode, enabled}></code>	Fetches rule configuration from the snapshot
<code>consumeNonce(string \$agentDid, string \$jti, int \$exp): array</code>	<code>{outcome, reason, httpStatus}</code>	Registers a single-use nonce for replay protection

Signature format (Stripe-style):

```
X-Trusted-Signature: t=<unix-timestamp>,s=<hmac-sha256-hex>
```

where the HMAC is computed over `"<timestamp>.<rawBody>"`.

AgentTokenVerifier

Class: `Trusted\AgenticCommerce\Service\AgentTokenVerifier`

Verifies Ed25519-signed agent JWT tokens:

1. Parses JWT header/payload (no library dependency, raw base64url decode)
2. Looks up the agent's public key from the snapshot (`getDidResolver`)
3. Verifies the Ed25519 signature using `sodium_crypto_sign_verify_detached` (or `paragonie/sodium_compat` fallback)
4. Validates `exp` , `iat` , `iss` , `aud` claims
5. Calls `consumeNonce` for replay protection

`CartSignals`

Class: `Trusted\AgenticCommerce\Service\CartSignals`

Extracts cart-level signals from a Magento quote for inclusion in the `/v1/rules/evaluate` payload:

- Cart total
- Item count
- Product category IDs
- Agent metadata from quote custom attributes

`AgentHistoryFetcher`

Class: `Trusted\AgenticCommerce\Service\AgentHistoryFetcher`

Fetches the agent's order history from the Trusted API to populate the `/v1/rules/evaluate` `agentHistory` context field.

10. Enforcement engine

Flow

```
sales_model_service_quote_submit_before
↓
CheckoutSubmitBefore::execute()
↓
Is this an agent order? (checkout session has trusted_agent_token)
↓ YES
AgentTokenVerifier::verify()
↓ VALID / INVALID / UNVERIFIED
EnforcementClient::evaluate({
  merchantId, agentId, orderContext,
  platform: "MAGENTO",
  installationId, timestamp
})
↓
ALLOW → continue (order is created normally)
BLOCK → throw LocalizedException (order is NOT created)
ESCALATE → freeze quote (is_active=0), stamp HITL flags, throw LocalizedException
```

Human checkouts also go through `evaluate()`, with `agentId` set to `null`, so your merchant rules apply to them too.

ESCALATE (R043 HITL)

When `evaluate()` returns `ESCALATE`:

1. `R043HitlGate::buildFreezePayload()` extracts rule code, reason, and evaluation ID
2. The quote is stamped with custom metadata:
 - `trusted_hitl_pending = 1`
 - `trusted_hitl_rule_code = <ruleCode>`
 - `trusted_hitl_reason = <reason>`
 - `trusted_hitl_evaluation_id = <evaluationId>`
3. Quote `is_active` is set to `0` (prevents customer-facing recapture)
4. A `LocalizedException` is thrown, so Magento does not create the order
5. The intent appears in the Trusted dashboard for merchant review

Failure mode

When the `evaluate()` call fails (transport error, non-2xx response or a rejected API base URL), the module first tries the offline safety valve. It checks the last rules snapshot against R014 (country check only), R018, R019, R020, R025, R027, R028, R029 and R030, and returns `BLOCK` on a match. If nothing matches, the failure mode decides:

Config value	Behavior
<code>enforce</code> (default)	Returns <code>BLOCK</code> . The order is rejected
<code>observe</code>	Returns <code>ALLOW</code> . The order proceeds

11. Webhook outbox

Delivery payload format

Each entry is delivered with `POST <api_base_url>/api/v1/webhook/magento/<connection_id>`. This is the body of an order-state event (`sales_order_save_after`):

```
{
  "event_id": "<uuid-v4>",
  "event_type": "order_created",
  "entity_id": 1,
  "increment_id": "000000001",
  "composite_id": "MAG:1",
  "store_view_code": "default",
  "updated_at": "2026-06-18T10:00:00+00:00",
  "payload": {
    "grand_total": 99.99,
    "status": "pending",
    "state": "new",
    "customer_email": null,
    "agent_did": null
  }
}
```

`customer_email` is always `null`, because the payload leaves out personal data. `agent_did` holds the verified agent identity for agent orders and is `null` for human checkouts.

Signature headers

```
Authorization: Bearer <integration token>
X-Trusted-Signature: <hex hmac-sha256>
X-Trusted-Timestamp: <unix seconds>
X-Trusted-Nonce: <32 hex characters>
X-Trusted-Webhook-Secret-Version: 1
```

HMAC input, keyed with the webhook secret: `"<timestamp>.<nonce>.<secretVersion>.<rawBody>"`

Retry schedule

The wait before each retry is $2 \text{ seconds} \times 2^{\text{retry_count}}$, capped at 1 hour, with $\pm 20\%$ random jitter. After 8 retries the entry's status is set to `dead`.

Dead entries are not retried. Use the Trusteed dashboard to replay dead webhooks manually if needed.

12. Agent token verification

Agent tokens are JWTs signed with Ed25519 (algorithm `EdDSA`, curve `Ed25519`).

Token claims

Claim	Type	Description
<code>iss</code>	string	Agent DID (e.g. <code>did:web:claude.ai</code>)
<code>sub</code>	string	Customer identifier
<code>aud</code>	string	Merchant ID
<code>exp</code>	unix timestamp	Token expiry (max 300 seconds from iat)
<code>iat</code>	unix timestamp	Issued at
<code>jti</code>	string	Single-use nonce (base64url, 16–128 chars)
<code>platform</code>	string	Agent platform (<code>claude</code> , <code>chatgpt</code> , etc.)

Verification outcomes

Outcome	Meaning
<code>VERIFIED</code>	Signature valid, claims valid, nonce not replayed
<code>INVALID</code>	Signature invalid, token expired, or nonce already used
<code>UNVERIFIED</code>	No token present or token could not be parsed

Public key resolution

Agent public keys are fetched from the enforcement snapshot (`GET /v1/rules/snapshot/<merchantId>`) as a JWK set. The snapshot is cached in-memory for the duration of the request to avoid repeated API calls.

13. MCP manifest

Endpoint: `GET /.well-known/mcp.json` Controller: `Trusted\AgenticCommerce\Controller\Wellknown\McpManifest` Builder: `Trusted\AgenticCommerce\Model\Manifest\Builder`

The manifest tells AI agents what capabilities your store exposes. It is signed with an Ed25519 key provisioned by Trusted.

Manifest fields

```
{
  "schema_version": "1.2",
  "merchant_id": "<merchant-id>",
  "store_url": "https://your-store.com",
  "connection_id": "<connection-id>",
  "platform": "magento",
  "capabilities": {
    "browse_catalog": true,
    "add_to_cart": true,
    "checkout": true,
    "payment_methods": ["x402", "stored_card"]
  },
  "mcp_endpoint": "https://api.trusted.xyz/mcp/<merchant-id>",
  "issued_at": "<iso8601>",
  "signature": "<jws-compact>"
}
```

14. Extension attributes

The module adds extension attributes to `Magento\Sales\Api\Data\OrderInterface`:

Attribute	Type	Description
<code>trusted_receipt_uri</code>	string	TrustReceipt URI
<code>trusted_receipt_status</code>	string	Receipt status. The module sets <code>signed</code> when it first stores the receipt URI

Defined in `etc/extension_attributes.xml`. Loaded/saved via `Plugin/Sales/OrderExtensionAttribute.php` and `Plugin/Repository/OrderRepositoryPlugin.php`.

15. API endpoints called

The module makes outbound HTTPS calls to the Trusted API. All calls require HTTPS and are validated by `ApiBaseUrlValidator` (SSRF guard).

Method	Path	When	Auth
POST	<code>/v1/rules/evaluate</code>	On every checkout attempt	<code>X-Trusted-Signature</code> (HMAC)
GET	<code>/v1/rules/snapshot/<merchantId></code>	Per-request cache miss	<code>X-Trusted-Signature</code> (HMAC)
POST	<code>/v1/agent-events/nonce-consume</code>	After token verification	<code>X-Trusted-Signature</code> (HMAC)
POST	<code>/api/v1/webhook/magento/<connection_id></code>	Webhook delivery	Bearer integration token and <code>X-Trusted-Signature</code> (HMAC)
POST	<code>/api/v1/internal/magento/lag-heartbeat</code>	Every minute (lag monitor)	Bearer integration token and <code>X-Trusted-Signature</code> (HMAC)
POST	<code>/api/v1/checkout-failures</code>	When a payment fails (R011 signal)	HMAC-signed payload
POST	<code>/api/v1/coupon-attempts-failed</code>	When a coupon is rejected	HMAC-signed payload
POST	<code>/api/v1/auth/introspect</code>	Setup Wizard token check	Bearer integration token
POST	<code>/api/v1/internal/magento/event</code>	Once, at installation (data patch)	Bearer integration token

Timeouts: 5 seconds for rule evaluation, snapshot and nonce calls, and 10 seconds for webhook delivery. The checkout-failure and coupon signals time out after 1.5 seconds, so they never hold up a checkout. TLS peer verification is always enabled (`CURLOPT_SSL_VERIFYPEER=true` , `CURLOPT_SSL_VERIFYHOST=2`). HTTP redirects are disabled (`CURLOPT_FOLLOWLOCATION=false`).

16. Security model

SSRF protection

`Model/Security/ApiBaseUrlValidator.php` validates the admin-configurable `api_base_url` against an allowlist of known Trusted API hostnames. Any attempt to change the API URL to a non-allowlisted host is rejected before any signed payload is sent.

TLS hardening

All outbound curl calls enforce:

- HTTPS scheme only (`CURLPROTO_HTTPS`)
- TLS peer and host verification
- No redirect following

HMAC signing

Calls to `POST /v1/rules/evaluate` are signed with HMAC-SHA256 in Stripe-style format:

`t=<timestamp>,s=<hex>`. The signing input is `"<timestamp>.<rawBody>"`. Webhook deliveries use the separate headers and signing input described in [Webhook outbox](#).

Webhook secret versioning

Every outbox entry stores the webhook secret version that was current when it was queued, and the delivery header `X-Trusted-Webhook-Secret-Version` carries that version. Rotate secrets by:

1. Updating the secret in Trusted dashboard
2. Pasting the new secret into Magento config
3. Incrementing `Webhook Secret Version`

Replay protection

Agent tokens include a `jti` (JWT ID) claim. The module calls `POST /v1/agent-events/nonce-consume` after verifying each token. A 409 response indicates replay, and the module treats the token as `INVALID`.

17. Console commands

Command	Class	Description
<code>trusted:check-webserver</code>	<code>Console/Command/CheckWebserver.php</code>	Validates that the <code>/well-known/mcp.json</code> endpoint is reachable from the server itself
<code>trusted:webhook:status</code>	<code>Console/Command/WebhookStatus.php</code>	Prints outbox statistics: pending, delivered, dead counts and oldest pending entry age

Usage:

```
bin/magento trusted:check-webserver
bin/magento trusted:webhook:status
```

18. Data patches

Patch Class	Purpose	Idempotent
<code>AddAgenticVisibleAttribute</code>	Adds the <code>is_agentic_visible</code> EAV attribute (boolean, default 1) to products and categories. The NLWeb product endpoint filters on it, so you can hide items from agents	Yes
<code>DisableBridgeOnHyva</code>	Detects Hyvä or PWA Studio themes and sets <code>trusted_general/features/webmcp_enabled = 0</code> to prevent storefront JS conflicts	Yes
<code>EmitInstallEvent</code>	Calls <code>POST /api/v1/internal/magento/event</code> to log the installation timestamp and Magento version in the Trusted dashboard	Yes

19. Di.xml wiring

Key dependency injection entries:

```
←!— EnforcementClient receives the SSRF validator via DI →  
<type name="Trusteed\AgenticCommerce\Service\EnforcementClient">  
  <arguments>  
    <argument name="urlValidator" xsi:type="object">  
      Trusteed\AgenticCommerce\Model\Security\ApiBaseUrlValidator  
    </argument>  
  </arguments>  
</type>  
  
←!— Outbox repository uses Magento ResourceModel pattern →  
<preference for="Trusteed\AgenticCommerce\Model\Webhook\OutboxRepositoryInterface"  
  type="Trusteed\AgenticCommerce\Model\Webhook\OutboxRepository"/>
```

See `etc/di.xml` and `etc/frontend/di.xml` for full wiring.

20. Logging

All module log entries are prefixed with `[trusteed]` and written to `var/log/system.log` (Magento default logger) at the following levels:

Level	Example messages
<code>debug</code>	Snapshot cache hit, nonce consume ACCEPTED
<code>info</code>	Outbox enqueue success, webhook delivered
<code>warning</code>	API timeout, SSRF guard rejection, HMAC verification mismatch, evaluate HTTP non-2xx
<code>error</code>	Fatal configuration missing, outbox drain failure after max retries

To enable debug logging:

```
bin/magento config:set dev/debug/debug_logging 1
```

Or set `MAGE_MODE=developer` in your environment.

Log grep shortcuts

```
# All Trusted entries
grep '\[trusted\]' var/log/system.log

# Outbox delivery problems only
grep '\[trusted\] webhook\|deliver\|dead\|retry' var/log/system.log

# Enforcement decisions
grep '\[trusted\] evaluate\|BLOCK\|ALLOW\|ESCALATE' var/log/system.log

# Agent token issues
grep '\[trusted\] token\|nonce\|verify\|INVALID\|REPLAY' var/log/system.log
```